Chapter 4

Linearizability

Linearizability is a method to reason about correctness of concurrent objects, i.e., objects that programmers can use in concurrent programs without the need of synchronization.[1] The goal of linearizability is to reuse sequential specifications of objects to define correctness for their concurrent counterparts. For instance, we can use the specification a queue that is meant to be used sequentially to establish the correctness of a queue implementation that can be used concurrently. Linearizability is commonly used to study the correctness of lock-free data structures.

First, we introduce a specific syntax for *concurrent executions, sequential executions and linearizations* that is conveniently tailored to reason about linearizability. The syntax is adapted from the graphical examples in [7].

4.1 Concurrent executions (to reason about linearizability)

Consider a program with two threads `A` and `B` that both access two shared objects `p` and `q`. Listing 4.1 is an example of a concurrent execution of this program.

```
A: -|  q.enq(x)  |-------|  p.enq(y)  |---->
B: -----|  q.deq(x)  |------|  p.deq(y)  |->
```

Listing 4.1: Example concurrent execution using the syntax to reason about linearizability.

Time is represented horizontally, either as `---` (nothing happens) or as `| <op> |` where some method call (`<op>`) is executed. The left `|` denotes the point in time when the method was invoked. The right `|` denotes the point in time when the response of the method call is received (in other words, when the method call finished). The width of `| <op> |` denotes the duration of the method call.

In listing 4.1, thread `A` first enqueues `x` in `q` and then `y` in `p`. Concurrently, thread `B` dequeues `x` from `q` and then `y` from `p`. In order to dequeue a specific element, e.g., `x`, an enqueue of the same element must have been executed before. For instance, in the execution above, it is not possible to execute `p.deq(y)` unless `p.enq(y)` has been executed first.

[1] Although the definition of concurrent object is very similar to that of thread-safe class (cf. chapter 6), we use different terms to distinguish their correctness properties.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-16

4.2 Sequential projections

A concurrent execution can be projected to several sequential executions. You can think of this projection as an order of execution (i.e., an interleaving) if the threads are executed concurrently in a computer with only one CPU. For the purposes of this chapter, real-time is irrelevant in sequential executions. We are only interested in the (sequential) order in which method calls happen. For example, the sequential executions in listing 4.2 are possible projections of the concurrent execution in listing 4.1.

```
<q.enq(x),p.enq(y),q.deq(x),p.deq(y)>
<q.enq(x),q.deq(x),p.enq(y),p.deq(y)>
```

Listing 4.2: Example of two possible sequential projections for the concurrent execution in listing 4.1.

Note that the syntax for sequential executions (projected from concurrent ones) is `<(op),(op),...>`. We use `<` to indicate the beginning of the execution, `>` to delimit the end, and `(op)` denotes an operation from the concurrent execution. As mentioned above, real-time is not part of the syntax, as only the order of execution is relevant.

At this point, a natural question is: *What are valid sequential projections for a given concurrent execution?* The answer to this question is the notion of *sequential consistency*.

4.3 Sequential consistency

In sequential programs, the compiler and CPU are allowed to re-order instructions as long as the result of the re-ordering does change the result of the program. For instance, below are three possible the orderings according to this rule:

```
<q.enq(x),p.enq(y),q.deq(x),p.deq(y)>
<p.enq(y),q.enq(x),p.deq(y),q.deq(x)>
<q.enq(x),p.deq(y),q.deq(x),p.enq(y)>
```

However, the last interleaving is problematic because `p.deq(y)` appears before `p.enq(y)`. This is inconsistent with the sequential specification of the behavior of a queue; as we cannot dequeue an element that has not been enqueued before. Therefore, it is common to require that all concurrent executions are *sequentially consistent*.

Sequential consistency is a property of concurrent executions [11]. In this section, we adapt it to focus on concurrent objects.

Definition 10 (Sequentially consistent execution of a concurrent object). An *execution of a concurrent object is sequentially consistent* iff

1. Method calls appear to happen in a one-at-a-time, sequential order.
2. Method calls appear to take effect in program order.

To determine if a concurrent execution is sequentially consistent, we must find at least one sequential projection that satisfies the specification of the object. More precisely,

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-16

Definition 11 (Conditions for a concurrent execution to be sequentially consistent). A concurrent execution of a concurrent object is *sequentially consistent* if there exist a sequential projection (cf. section 4.2) of method calls where:

1. Operations happen one-at-a-time.
2. Program order is preserved (for each thread).
3. The execution mapping satisfies the specification of the object.

Intuitively, to determine whether a concurrent execution is sequentially consistent, we can imagine that we can slide the blocks `| <op> |` forward and backward in time, but with the constraint that two consecutive blocks within a thread cannot swap positions (i.e., intra-thread program order must be preserved). Using this "sliding rule", if we can find a sequential projection that satisfies the sequential specification of the object, then the execution is sequentially consistent.

Example 1. We recall here the concurrent execution in listing 4.1 and show that it is sequentially consistent.

```
A: -|  q.enq(x)  |-------|  p.enq(y)  |---->
B: -----|  q.deq(x)  |------|  p.deq(y)  |->
```

We can slide the blocks `| <op> |` as follows:

```
          ⟵                          ⟵
A: -|  q.enq(x)  |----------------|  p.enq(y)  |---------------->
                        ⟶                          ⟶
B: ----------------|  q.deq(x)  |----------------|  p.deq(y)  |->
```

which leads to the sequential projection

```
<q.enq(x),q.deq(x),p.enq(y),p.deq(y)>
```

This sequential execution is legal with respect to the specification of a queue. That is, we enqueue every element before we dequeue it.

The sequential projection above is suffient to show that the concurrent execution in listing 4.1 is sequentially consistent (cf. definition 11). Nevertheless, there exist another sequential projections that satisfy the specification of a queue.

```
                          ⟵
A: -|  q.enq(x)  |---|  p.enq(y)  |--------------------------------->
                                          ⟶
B: ---------------------------------|  q.deq(x)  |---|  p.deq(y)  |->
```

which results in a sequential projection that satisfies the sequential execution of a queue:

```
<q.enq(x),p.enq(y),q.deq(x),p.deq(y)>
```

Any of the two sequential projections is suffient to show that the concurrent execution in listing 4.1 is sequentially consistent. □

Finally, we lift the definition of sequential consistency from executions to concurrent objects as follows:

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-16

Definition 12 (Sequentially consistent concurrent object). A *concurrent object is sequentially consistent* if all concurrent executions of method calls are sequentially consistent.

Unfortunately, *sequential consistency is not a compositional property.* That is, concurrent executions that involve two sequentially consistent objects may not be sequentially consistent. We demonstrate the lack of compositionality with an example.

Example 2. We consider a concurrent execution of two stacks `p` and `q`. Recall that stacks follow a *Last-In-First-Out* (LIFO) policy. This is our sequential specification for the stack. The concurrent execution is as follows:

```
A: ---|  q.push(y)  |---|  q.push(x)  |---|  p.push(y)  |->
B: ------|  p.pop(y)  |------|  q.pop(y)  |--------------->
```

Listing 4.3: Compositionality counterexample for sequentially consistent concurrent objects.

We show that there is no sequential projection of the concurrent execution that satisfies the sequential specification of a stack.

We start by observing that, if we consider operations that only involve a single object (either `p` or `q`), then those executions are sequentially consistent.

```
// Only p
A: -----------|  p.push(y)  |->
B: ------|  p.pop(y)  |------->

// Only q
A: ---|  q.push(y)  |---|  q.push(x)  |->
B: ------------|  q.pop(y)  |----------->
```

For the `Only p` execution we have the sequential projection `<p.push(y),p.pop(y)>`, and for the `Only q` execution we have the sequential projection `<q.push(y),q.pop(y),q.push(y)>`. Both sequential projections satisfy the sequential specification of a stack.

Now we show, graphically, that there is no sequential projection for the composed concurrent execution in listing 4.3 that satisfies the sequential specification of the stack. In order for the sequential specification to hold, we must ensure the following properties: i) `p.pop(y)` is executed after `p.push(y)`; and ii) `q.pop(y)` is executed after `q.push(y)` and `q.push(x)` is executed before `q.push(y)`.

If we attempt to enforce property i), then we obtain the following "slided" concurrent execution:

```
A: -|  q.push(y)  |---|  q.push(x)  |---|  p.push(y)  |-------------------------------->
                                                   ⟶
B: ---------------------------------------------------|  p.pop(y)  |---|  q.pop(y)  |->
```

However, the sequential projection executes `q.push(x)` after `q.push(y)`, as intra-thread program order must be preserved. This violates property ii), as `q.pop(y)` cannot be executed if `x` was the latest element pushed into the tack.

The other option is to attempt to enforce property ii) as shown below.

```
            ⟵                                ⟶
A: ------|  q.push(y)  |----------------|  q.push(x)  |---|  p.push(y)  |--->
B: -|  p.pop(y)  |-------|  q.pop(y)  |------------------------------------>
```

Two sequential projections can be obtained from the above:

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-16

```
<p.pop(y),q.push(y),q.pop(y),q.push(x),p.push(y)>
<q.push(y),p.pop(y),q.pop(y),q.push(x),p.push(y)>
```

In either of them `p.pop(y)` is executed before `p.push(y)`, which violates property i).

This reasoning (informally but systematically) shows that it is not possible to obtain sequential projection that preserves the sequential specification of a stack.

All in all, this counterexample shows that sequential consistency is not compositional, as we have shown that combining two sequentially consistent objects into a single concurrent execution makes the execution not sequentially consistent. □

The lack of compositionality of sequential consistency hampers the possibility of using it for modular reasoning/design of concurrent objects. Linearizability addresses this problem.

4.4 Linearizability

Linearizability extends sequential consistency by requiring that the real-time order of executions is preserved. To this end, there must be a moment between invocation and response of method calls where the method takes effect, called a *linearization point*. In lock-free data structures, linearization points typically (but not always) correspond to compare-and-swap (CAS) operations. For data structures that use locks, any point in the critical section protected by the lock can be selected as a linearization point.

For instance, consider the following implementation of the `dequeue()` method of the Michael-Scott lock-free queue.

```
public T dequeue() {
  while (true) {
    Node<T> first = head.get();
    Node<T> last = tail.get();
    Node<T> next = first.next.get(); // D3
    if (first == head.get()) { // D5
      if (first == last) { // D6
        if (next == null) // D7
          return null;
        else
          tail.compareAndSet(last, next);
      } else {
        T result = next.item;
        if (head.compareAndSet(first, next)) // D13
          return result;
      }
    }
  }
}
```

Statements D3 and D13 define the instant in the execution when the method call "takes effect" and hence may cause inconsistencies. They are the linearization points of this method. When statement D13 is executed, the dequeuing of an element from an non-empty queue is effectively completed. Similarly, D3 marks the moment when dequeuing from an empty queue takes place. We discuss linearizability of the Michael-Scott queue in section 4.4.3. Before that, we start by defining linearizability for concurrent executions and concurrent objects.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-16

4.4.1 Linearizability of concurrent executions

Linearizability extends the definition of sequential consistency (cf. definition 10) by requiring that each method call appears to take effect instantaneously at the linearization point(s).

Definition 13 (Linearizable execution of a concurrent object). An execution of a concurrent object is *linearizable* iff

1. It is sequentially consistent.
2. Each method call appears to effect at some point between its invocation and response (i.e., at a linearization point).

Note that it trivially follows from definition 13 that any linearizable execution is also sequentially consistent.

To determine whether a concurrent execution is linearizable, we must find linearization points that produce a sequential projection that satisfies the sequential specification of the object.

Definition 14 (Condition for a concurrent execution to be linearizable). A concurrent execution of a concurrent object is *linearizable* iff there exist linearization points that produce a sequential projection that satisfies the sequential specification of the object.

Note that this definition only refers to linearization points between invocation and response of method calls. Thus, when checking linearizability of a concurrent execution, we cannot "slide" `| <op> |` blocks. This restriction corresponds to preserving the real-time order of the concurrent execution (which is required by linearizability). Consequently, to determine whether a concurrent execution is linearizable, we can only select linearization points between invocation and response of method calls, i.e., in between `| <op> |` blocks.

To analyze linearizability of concurrent executions, we extend our syntax with the symbol `*` to mark the linearization points. As before, we illustrate its use by example.

We show that the concurrent execution in listing 4.1 is linearizable. We start by marking linearization points for each of the method calls in the execution.

```
A: -| *q.enq(x)  |-------|* p.enq(y)  |---->
B: -----|  q.deq(x) *|------|  p.deq(y)* |->
```

Listing 4.4: Example linearization points for execution in listing 4.1.

At the level of executions (i.e., without looking at the implementation of the queues), we can select linearization points anywhere in between `| <op> |`. Linearization points can also be placed within the text of the operation, e.g., `| r.e*nq(y) |`.

The selected linearization points produce a sequential projection (known as a *linearization* in this context). For the example above (cf. listing 4.4), we obtain the following linearization:

```
A: -| *q.enq(x)  |-------|* p.enq(y)  |----->
B: ---:-|  q.deq(x) *|----:-|  p.deq(y)* |-->
      :             :     :           :
   < q.enq(x), q.deq(x), p.end(y), p.deq(y) >
```

Listing 4.5: Linearization of the concurrent execution in listing 4.1 given the linearization points in listing 4.4.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-16

This linearization is consistent with the sequential specification of a queue. Therefore, we have shown that the concurrent execution is linearizable given the selected linearization points. The linearization above is sufficient to show that the concurrent execution is linearizable.

Note that, unlike for sequential consistency, the linearization above is the only possible linearization that satisfies the specification of a sequential queue. As mentioned earlier, when reasoning about linearizability, we cannot "slide" method calls. Hence, we must pick linearization points where the first enqueue/dequeue of `x` occurs and afterwards the enqueue/dequeue of `y`.

If we had a *different* concurrent execution where the duration of the `p.enq(y)` and `q.deq(x)` increases to the point that they overlap, then we can find an alternative linearization.

```
A: -| *q.enq(x)  |--|*      p.enq(y)  |--------->
B: ---:-|  q.deq(x)  :   *|------| * p.deq(y) |->
       :              :   :         :
   < q.enq(x), p.enq(y), q.deq(x), p.deq(y) >
```

This concurrent execution has two possible linearizations, depending on the selection of linearization points.

Finally, consider this execution:

```
A: -| q.enq(x) |------| q.deq(y)|---->
B: --------------| q.enq(y) |-------->
```

This concurrent execution is not linearizable. The reason is that we cannot select linearization points so that `y` is enqueued before `x`, which violates the FIFO policy of the queue.

4.4.2 Linearizable objects

As before, we can lift the definition of linearizability from executions to concurrent objects.

Definition 15. A *concurrent object is linearizable* iff all concurrent executions of method calls are linearizable.

Unlike sequential consistency, *linearizability is a compositional property.* If we have shown that two objects are linearizable, any concurrent executions involving (only) mehotds calls on these two objects is linearizable. This is a very useful property as it enables modular correctness reasoning. The propertines allows to split reasoning into single objects that can then be safely composed.

4.4.3 Reasoning about linearizability on object implementations

In this section, we illustrate how to analyze linearizability of concurrent objects implementations. We do not target formal proofs. Note that proving whether an object is linearizable requires looking all possible concurrent execution of the method calls (cf. definition 15), which is a very tedious task. Instead, we focus on a more pragmatic approach where we argue (informally but systematically) about the behavior of the linearizations. More precisely, to argue that an object is linearizable, we start by identifying all linearization points (typically placed where CAS operations occur). We then analyze the possible states of the data structure when reaching the linearization point.

In what follows, we describe two examples of linearizability analysis of concurrent objects (concretely, lock-free data structures). The first example is an implementation of a FIFO queue called `WaitFreeQueue` (that turns out to not be linearizable), and the second is the Michael-Scott queue (which is known to be linearizable).

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-16

WaitFreeQueue

Consider the following Java implementation of a `WaitFreeQueue` from [7, page 52]. We have marked every line of method implementations with an operation identifier to discuss linearization points.

```
class WaitFreeQueue<T> {
  int head = 0, tail = 0;
  T[] items;

  public WaitFreeQueue(int capacity) {
    items = (T[]) new Object[capacity]
  }

  public void enq(T x) throws Exception {
    if (tail - head == items.length) // E1
      thrown new Exception(); // E2
    items[tail % items.length] = x; // E3
    tail++; // E4
  }

  public void deq() throws Exception {
    if (tail - head == 0)  // D1
      thrown new Exception(); // D2
    T x  = items[head % items.length]; // D3
    head++; // D4
    return x;
  }
}
```

Recall that each method has its own linearization points. Thus, we discuss the linearization points of each method separately. Recall also that a method can have more than one linearization point depending on the possible outcomes of execution the method.

The linearization points for `enqueue` are: E4 and E1. E4 is the point when an enqueue is completed if the queue is not full, and E1 is the point when an enqueue is completed if the queue is full.

The linearization points for `dequeue` are: D4 and D1. D4 is the point when a dequeue is completed if the queue is not empty, and D1 is the point when a dequeue is completed when queue is empty

We show that the implementation of `WaitFreeQueue` is not linearizable. To this end, we must find an execution that is not lineariable; i.e., a counterexample for linearizability.

Consider the execution: E1,E3,E1,E3,E4,E4. Here E1,E3 are excuted twice before incrementing `tail`. As a result, the second execution of E3 overwrites the first. This leads to the following linearization: `<q.enq(x),p.endq(y),q.deq(y)>`. However, this linearization violates the specification of a FIFO queue. Consequently, the `WaitFreeQueue` is *not* linearizable.

Michael-Scott Queue

The Michael-Scoot queue is (yet another) implementation of a FIFO queue. Below is a Java implementation for the enqueue/dequeue methods. As before, we have assigned identifiers to relevant lines of code in method implementations to discuss linearizability.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-16

```
public void enqueue(T item) {
  Node<T> node = new Node<T>(item, null);
  while (true) {
    Node<T> last = tail.get();
    Node<T> next = last.next.get();
    if (last == tail.get()) { // E7
       if (next == null)  { // E8
         // In quiescent state, try inserting new node
         if (last.next.compareAndSet(next, node)) { // E9
        // Insertion succeeded, try advancing tail
        tail.compareAndSet(last, node);
        return;
         }
       } else
       // Queue in intermediate state, advance tail
       tail.compareAndSet(last, next);
    }
  }
}

public T dequeue() {
  while (true) {
    Node<T> first = head.get();
          Node<T> last = tail.get();
    Node<T> next = first.next.get(); // D3
          if (first == head.get()) { // D5
            if (first == last) { // D6
              if (next == null) // D7
          return null;
        else
          tail.compareAndSet(last, next);
      } else {
        T result = next.item;
        if (head.compareAndSet(first, next)) // D13
          return result;
      }
    }
  }
}
```

We start by identifying the linearization points for the two methods.

Enqueue has one linearization point: E9. This linearization point corresponds to a successful enqueue. If E9 returns true, then it is guaranteed that `item` is added to the queue.

Dequeue has two linearization points: D3 and D13. When executing dequeue on an empty queue, D3 determines the moment where the execution of the method is guaranteed to return null. The reason is that if D3 returns null, then the condition in D7 is guaranteed to evaluate to true, and, consequently, the dequeue method will return null. D13 marks the moment when a successful dequeue occurs in a non-empty queue.

Below we argue about linearizability of the implementation by (attempting to) cover all execution branches (i.e., the if-statements branches) of each method. The goal is to systematically discuss all possible executions of the queue.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-16

Linearizability of enqueue Our analysis focuses on the 3 conditional statements of the queue: E7, E8 and E9; as these determine the possible states of the queue.

If two threads execute enqueue concurrently before `tail` and `next` are updated, then only one of them succeeds in executing E9 (and possibly updates the `tail`). The other fails and repeats the enqueuing.

If a thread executes enqueue after another thread updated the `tail`, then E7 fails and it repeats the enqueue.

If a thread executes enqueue after another thread updated `next`, then E8 fails, the thread tries to advance the tail, and it restarts the enqueue.

All in all, this behavior complies with the specification of a queue when two enqueues occur concurrently.

Linearizability of dequeue Here the analysis focuses on the statements: D5, D6, D7, D13 (and also E9).

If two threads execute dequeue concurrently before the `head` is updated (D5 succeeds for both) and the queue is not empty (D6 fails), then D13 succeeds for only one of them. The other restarts the dequeue.

If a thread executes dequeue after another thread updated the `head`, then D5 fails and it restarts the dequeue.

If a thread executes dequeue while another thread executed enqueue (E9) and before the enqueuing thread updates the `tail`, then D7 fails. Then, the dequeuing thread tries to update the `tail` and restarts the dequeue.

The reasoning covers the different situations of concurrent dequeues. Additionally, we analyze a situation where a dequeue and enqueue occur concurrently; which leads to a concurrent access on `tail`. In all cases, the specification of the queue is respected.

The reasoning above provides evidence that all executions of the Michael-Scott queue satisfy the specification of a FIFO queue. Consequently, we conclude that this implementation of the Michael-Scott queue is linearizable.

4.5 Progress conditions for non-blocking concurrent objects

Non-blocking concurrent objects (i.e., concurrent object that do not use locks for synchronization) can be categorized in three different progress conditions:[2] *wait-free*, *lock-free* and *obstruction-free* [7]. Determining what progress conditions apply to a concurrent object is important, as it allows us to understand the expected behavior of the object and how it will perform under different levels of concurrency (e.g., high vs low thread contention). This categorization complements correctness reasoning. For example, linearizability allows us to show that a concurrent object behaves as its sequential counterpart, but it does not say anything about how efficiently the object behaves under high thread contention. In what follows, we describe each non-blocking progress condition.

We start with wait-freedom, as it is the progress condition that imposes the strongest progress requirements.

[2]These are the counterparts of deadlock-freedom (definition 7) and starvation-freedom (definition 8) that we discussed in chapter 1 for blocking concurrent objects.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-16

Definition 16 (Wait-freedom). A method of a concurrent object implementation is *wait-free* if every call finishes its execution in a finite number of steps.

This progress condition requires that any number of (concurrent) method calls finish in a number of steps regardless of the behavior of any of the threads in the system. That is, every thread must complete its method execution independently of the behavior of other threads. This is a strong and useful progress condition. Since termination of wait-free methods does not depend on the execution of other concurrent threads, we might expect reliable performance even under high thread contention.

Example 3. Consider the `ReadyInteger` in listing 4.6. This is a simple integer class that can only be incremented if a Boolean flag `ready` is set to `true`. We use a non-blocking implementation that only uses CAS operations from `Atomic` classes.

The method `readyToUse` is wait-free. To understand why, consider two threads (`t1` and `t2`) that concurrently execute `readyToUse`. The possible executions are: `t1(rCAS),t2(rCAS)` and `t2(rCAS),t1(rCAS)`; where `rCAS` denotes the CAS operation on `ready` performed within the `readyToUse` method. Note that in both executions each thread executes a single CAS operation. Furthermore, the behavior of `t1` does not affect execution of `t2` and vice versa; as the outcome of the CAS operation does not influence the execution of the method. These properties hold for any number of threads in the execution. Thus, we can conclude that the execution of `readyToUse` completes in one step (as it contains only one atomic instruction) for any number of threads; without making any assumptions on the behaviors of other threads.

On the contrary, the method `increment` is *not* wait-free. Consider the following execution where `t1` and `t2` execute `increment` in an infinite loop:

```
[t1(iGet),t2(iGet),t2(iCAS_succeed),t1(iCAS_fail)]*
```

We use `[...]*` to denote that the content within the squared brackets is executed arbitrarily many times. This execution shows the case where `t1` tries to increment the integer but fails (denoted as `t1(iCAS_fail)`) due to `t2` succeeding (`iCAS_succeed`). In this execution, `t2` always finishes the execution of increment in a finite number of steps, 2 steps. But `t1` would never finish in a finite number of steps. As usual, to show that a property does not hold, it is sufficient to find a counterexample. Thus, the execution above is sufficient to conclude that `increment` is not wait-free. □

Lock-freedom relaxes wait-freedom by only requiring that some thread completes the method execution in a finite number of steps.

Definition 17 (Lock-freedom). A method of a concurrent object implementation is *lock-free* if at least one method call finishes in a finite number of steps.

Lock-freedom is the most common progress condition used in non-blocking concurrent objects. It is typically achieved by using the standard optimistic concurrency loop where a thread reads the state of the object, tries to update it using a CAS operation, and, if it succeeds, it terminates, otherwise it restarts the loop. In this loop, a thread not being able to terminate can only be caused by another thread having successfully performed the method call; as required by lock-freedom.

Example 4. Consider again the method `increment` in listing 4.6. This method is lock-free. First, note that the only part of the code that can result in a infinite execution is the `do-while` loop; as the `if` statement to check the value of `ready` always finishes in a finite number of

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-16

```
class ReadyInteger {
  private final AtomicBoolean ready = new AtomicBoolean(false);
  private final AtomicInteger value = new AtomicInteger(0);

  public void readyToUse() {
    ready.compareAndSet(false,true); // (rCAS)
  }

  public boolean increment() {
    if (!ready.get())
      return false;
    int currentValue;
    do {
      currentValue = value.get(); // (iGet)
    } while (!value.compareAndSet(currentValue, currentValue+1)); // (iCAS)
    return true;
  }
}
```

Listing 4.6: Lock-free integer class that requires setting a Boolean flag to allow for increments.

steps. A thread remains in the `do-while` loop if the `iCAS` operation returns false. The only reason why this can occur is if a different thread successfully executed `iCAS`, as there is no other method in the class that modifies `value`. Thus, we can conclude that the method is lock-free.

The wait-freedom counterexample execution in example 3 illustrates the behavior described above. Thread `t1` never finishes the execution of increment, but only because `t2` always succeeds. That is, there exist a thread that completes the execution of the method in a finite number of steps. □

Obstruction-freedom is the weakest progress condition. It only requires that a thread finishes the execution of a method, assuming that at some point in the execution it will run in isolation; i.e., no other thread will interfere with the execution.

Definition 18 (Obstruction-freedom). A method of a concurrent object implementation is *obstruction-free* if every method call finishes in a finite number of steps, provided that from some point in the execution it is executed in isolation.

This property can be seen as a sanity check that ensures that the execution of the method terminates when executed in isolation.

Example 5. We show that all methods in listing 4.6 are obstruction-free.

Recall that we showed that `readyToUse` is wait-free (cf. example 3). We showed that the method execution must finish in a finite number of steps, independently of the behavior of other threads. In particular, it must finish in a finite number of steps when there are no other threads executing at all, as assumed by obstruction-freedom.

Similarly, in example 4 we showed that `increment` is lock-free. That is, in a concurrent execution of method calls, at least one of them must finish in a finite number of steps. If the thread is executed in isolation (as stated in obstruction-freedom), then the execution only contains one thread, and it is that thread that finishes in a finite number of steps. More

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-16

concretely, the `iCAS` operation never fails if only one thread is executing the method. □

Generally, progress conditions form a hierarchy where wait-freedom is the strongest condition and obstruction-freedom the weakest:

$$\textit{wait-freedom} \rightarrow \textit{lock-freedom} \rightarrow \textit{obstruction-freedom}$$

That is, wait-free methods are lock-free, lock-free methods are obstruction-free and wait-free methods are obstruction-free.

A remark on terminology As you might have noticed at this point, the term *lock-free data structure* (or class) is not necessarily related to the use of locks. In this chapter, we have used the term "non-blocking concurrent object" to refer to implementations that do not use locks. The reason is that there exist non-blocking objects that do not enforce lock-freedom (i.e., they may enforce wait-freedom or obstruction-freedom).

A practical remark As discussed above, the strongest and most reliable progress condition is wait-freedom. However, in practice it is difficult to achieve. Consequently, most implementations of non-blocking objects are lock-free (e.g., Treabor's stack or Michael-Scott queue). Interestingly, it has been shown that, under certain scheduling conditions, lock-free implementations usually behave as wait-free [1]. In practice, this means lock-free implementations are often as reliable as wait-free ones.

© Copyright 2026 Raúl Pardo, Peter Sestoft, Jørgen Staunstrup. All rights reserved. V 2026-10-16